



PEPS & PINK PEPS

Positional Encoding Projected Sampling

Guillaume Perez, Janarбек Matai, Takahiro Harada

2026 May



Motivation

- Implicit neural representations (INRs)
 - Maps coordinates to signals (color, distance, etc.)
 - Used for images, textures, SDFs, NeRF-like models
- MLPs
 - Not good at learning fine detail
 - Spectral bias — they learn low frequencies first
- Input encoding
 - Critical to represent high-frequency details
 - e.g., Positional Encoding, grid encoding
- Can we improve input encoding further?



Existing Encoders

- Positional (Absolute)
 - Encode scalar x as

$$\text{PE}(x, 2i) = \sin x \phi_i$$

$$\text{PE}(x, 2i + 1) = \cos x \phi_i$$

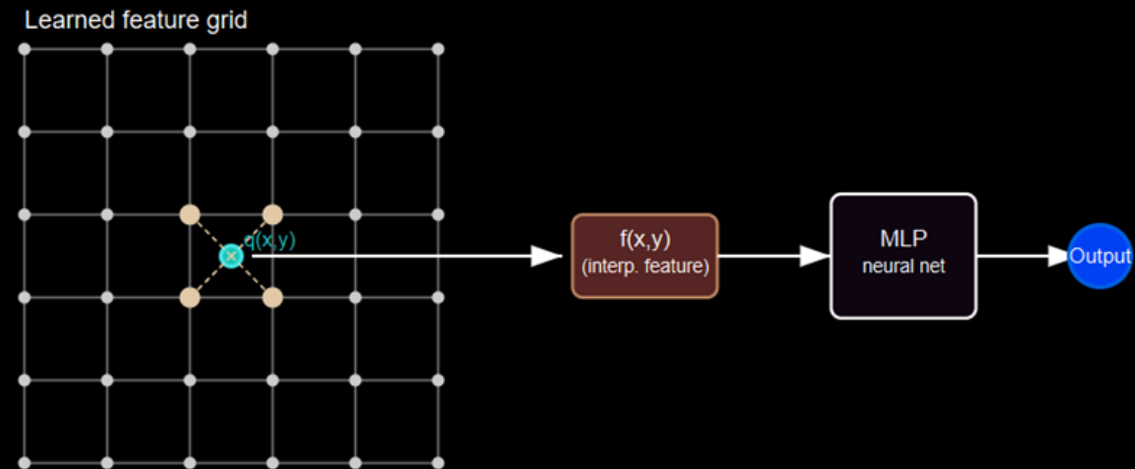
- Using fixed sinusoidal frequencies
 - Variants: fixed, learned, randomly sampled, etc.

$$\text{PE}(x) = [\cos(2^0 \pi x), \sin(2^0 \pi x), \dots, \cos(2^{n-1} \pi x), \sin(2^{n-1} \pi x)],$$

- Grid (Learned)

$$f_\theta : x \rightarrow e$$

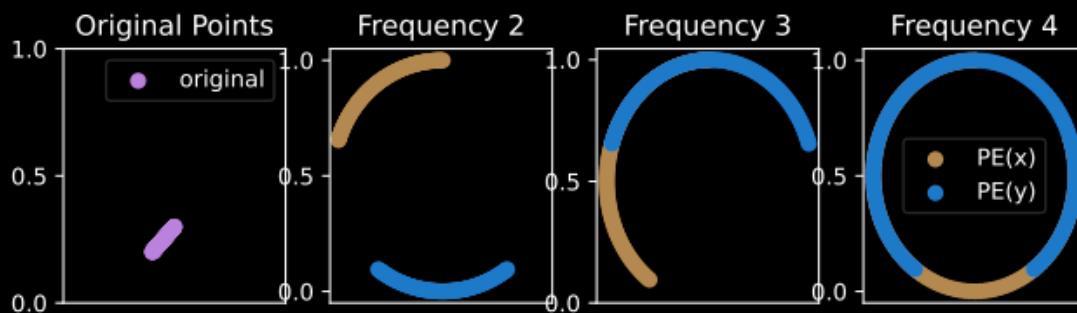
- Any function that takes the coordinates
 - And returns an embedding
- For implicit neural representations (INRs)
 - Discrete grid of learned feature vectors
 - Sample and interpolate features at input coordinates
 - Variants: multi-resolution grids, hash grids, etc.





Limitation of Existing Encoders

- Identity:
 - No encoding
 - Coordinate directly feed to the MLP
 - Issue: spectral bias of MLP
 - MLPs struggle to represent high-frequency signals from low-dimensional inputs
- Positional Encoding
 - Fixed spectrum, cannot adapt to data
 - Independent axis wise
- Grid / Hash encoders:
 - Need high resolution → many parameters
 - Susceptible to artifacts: aliasing, blocky patterns, hash collisions
 - Limited control over how frequencies are represented
- The resolution MLP can express is defined when the input encoder is defined
 - True for both positional & grid

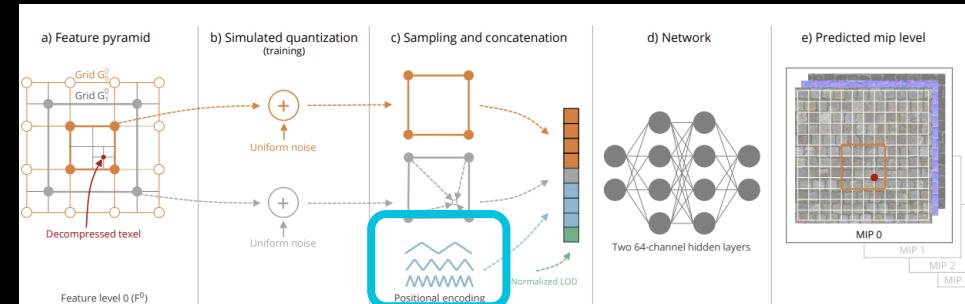


Showing where line of points are mapped after positional encoding

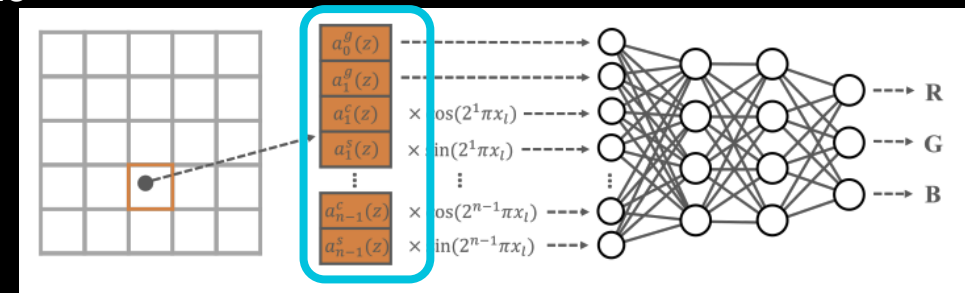


Previous Works

- Using grid and positional encoders together
- Add positional encoding on top of grid encoding
 - Random-Access Neural Compression of Material Textures, 2023
 - `Cat( grid, positional )`



- Grid encoding computes scaling factor for each term in positional encoding
 - Local Positional Encoding for Multi-Layer Perceptrons, 2023
 - `grid * positional`

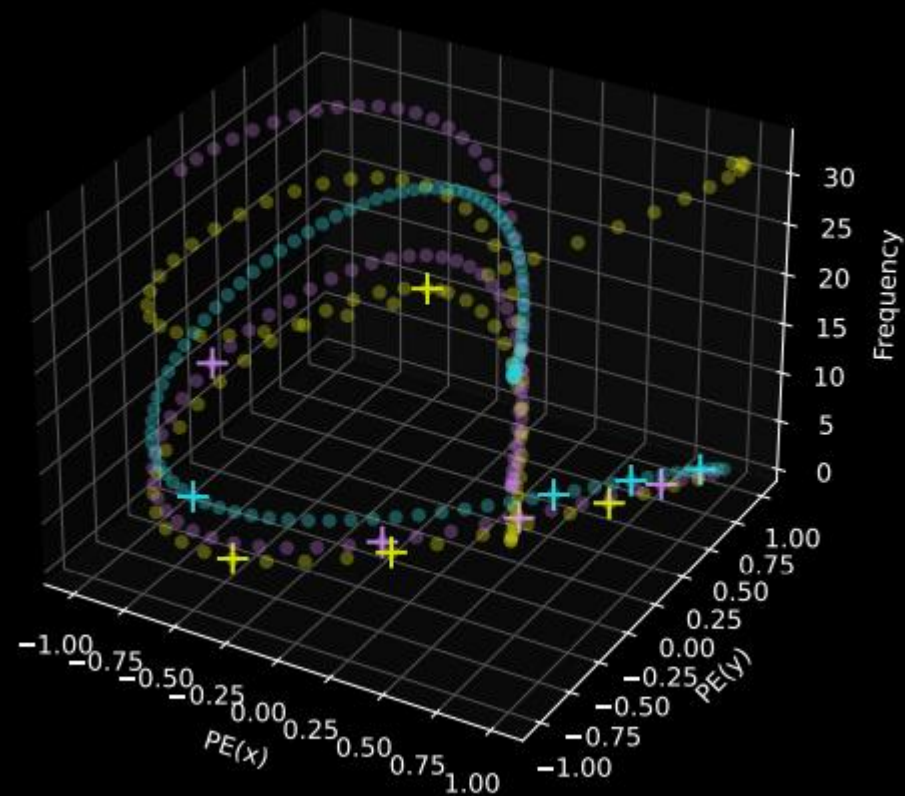


PEPS: Positional Encoding Projected Sampling



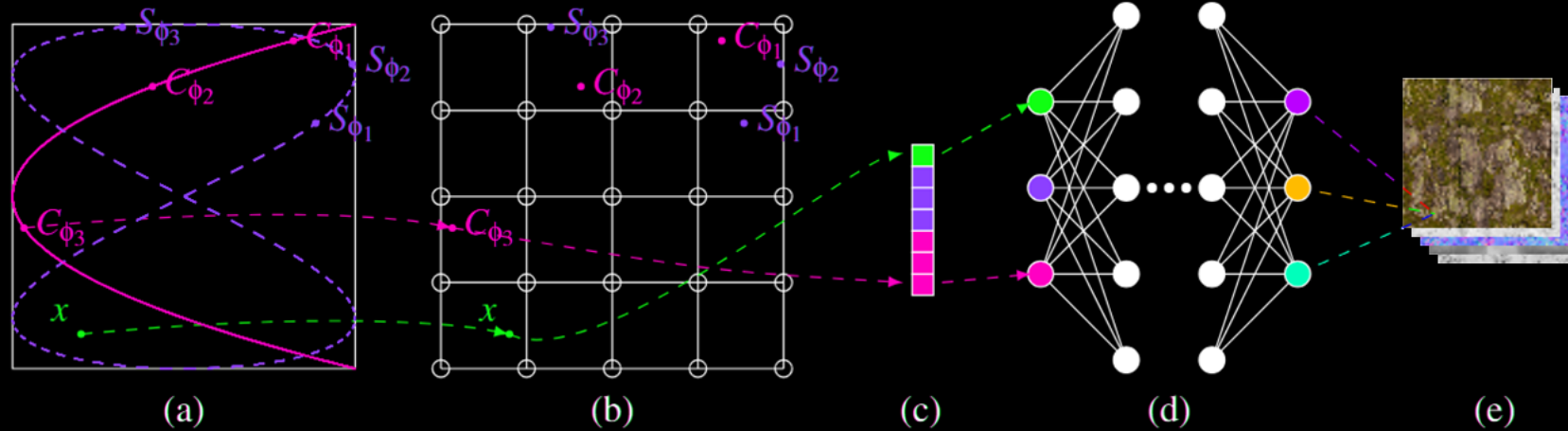
Key Idea: Positional Encoding as Trajectories

- Positional encoding generates a unique trajectory for (x)
 - Lissajous curves (high dim)
- Regular grid encoding
 - Sample the grid at X
- PEPS
 - Generate a unique curve from X
 - Sample the grid using the curve (Instead of sampling using X only)





PEPS: Positional Encoding Projected Sampling



- Wrapper around **encoders**
- Input coordinate (x)
 - (a) Project (x) to a set of points on its PE curve (via sinusoidal PE)
 - (b) Encode (x) and each projected point using a base **encoder**
 - (c) Aggregate all latent vectors
 - (d) Feed aggregated latent vector into MLP to obtain output signal (e)

- General Formula:

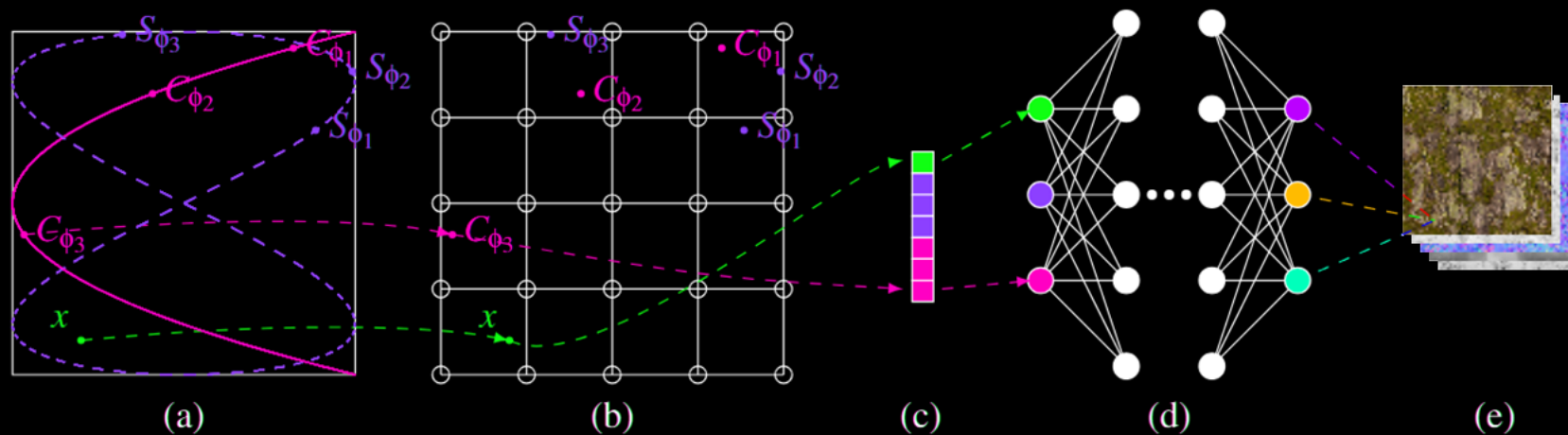
$$M\left(A(E_1(P_1^x), \dots, E_{2L+1}(P_{2L+1}^x)), \delta\right),$$

- M: MLP (delta are additional parameters), A: Aggregation Function, P: the original and projected points



Input Dimension Reduction

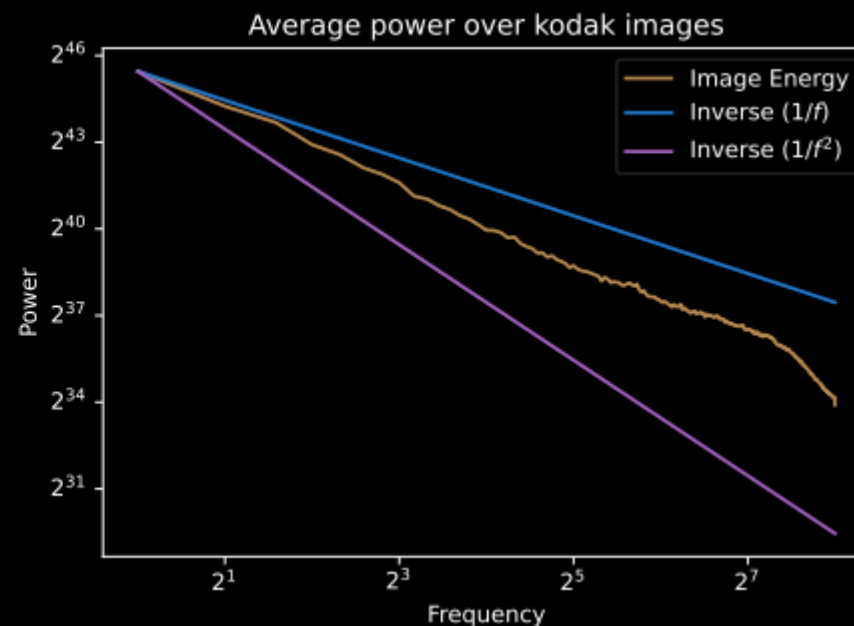
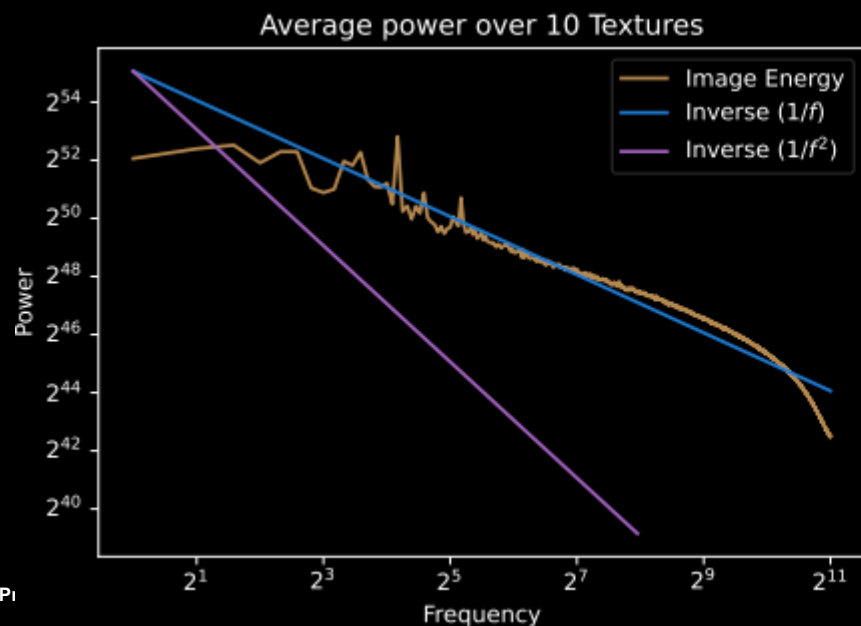
- PEPS
 - Samples grid multiple times
 - Concatenate feature vectors
- The input dimension of the MLP increases (7x in the image below)
- Runtime overhead
- Question
 - Can we reduce the input dim somehow?
 - What should we remove?





Power Spectral Density for Natural Processes

- Used the nature of images
- **Pink** Power Spectral Density (PSD)
 - The power spectral density decreases as the frequency increases
 - $1/f^a$
 - Confirmed this is satisfied with the test data
- This indicates we can reduce the latent vector allocation for higher frequency information
- Regular PEPS == **White** PEPS (White PSD)

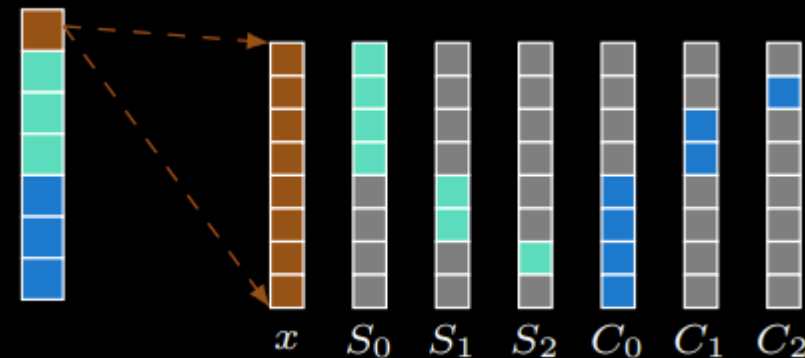




Pink-PEPS Aggregator

- Design an aggregation function controls the power allocated to each frequency
- 16 features over 5 frequencies
 - $[0^{\text{th}}, 1^{\text{st}}, 2^{\text{nd}}, 3^{\text{rd}}, 4^{\text{th}}] = [8, 4, 2, 1, 1]$

0	0	1	1
0	0	1	1
0	0	2	3
0	0	2	4



PEPS: **56** = 8×7 , Pink-PEPS: **22** = $8 + 2 \times 4 + 2 \times 2 + 2 \times 1$

Algorithm 1 Full Grid-Pink-PEPS algorithm

Require: $x \in \mathbb{R}^d, E$: grid encoder, $L \in \mathbb{N}$

```

 $g \leftarrow E(x)$ 
for  $i = 1, \dots, L$  do
   $l^{S_i} = E(\sin x \phi_i)$ 
   $l^{C_i} = E(\cos x \phi_i)$ 
   $g.append(l^{S_i}_{-G_i:-G_{i-1}})$ 
   $g.append(l^{C_i}_{G_{i-1}:G_i})$ 
end for
return  $g$ 

```

G is defined in the paper to extract the sub-vectors according to the image above

Results



Questions

- Does grid-PEPS break the resolution as core parameter for improvements?
- Does PEPS improve reconstruction quality?
- Does it reduce model size?
- Does it reduce artifacts and improve details?



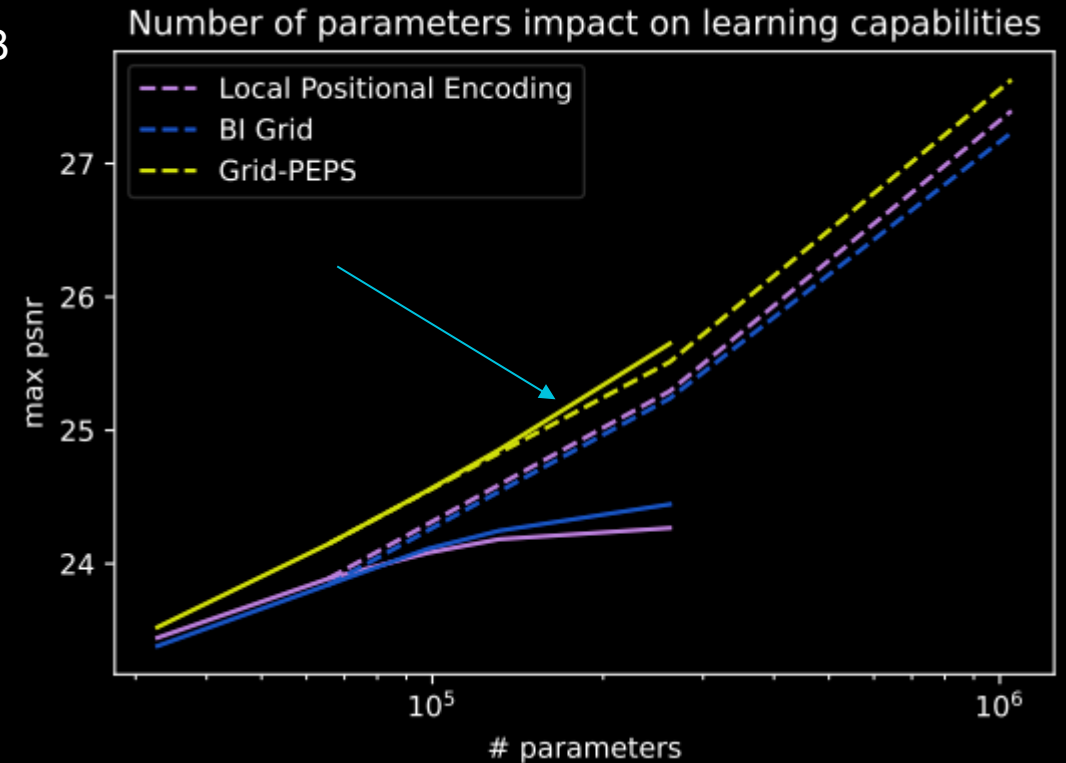
Evaluation

1. Implicit Image Representation
2. Texture Compression
3. Signed Distance Functions (SDFs)



Implicit Image Representation

- Problem:
 - Given a coordinate $x \in [0, 1]^2$, the function $P(x)$ returns the RGB values $(r, g, b) \in \mathbb{R}^3$
- Settings:
 - Learning 4K images (roughly 10^8 values)
 - With models having between 10^4 to 10^6 parameters
 - grid resolutions between 16 to 128
 - feature vector dimensions between 8 to 64
- Hypothesis:
 - Any additional parameters should improve the model
- This plot:
 - Only the resolution increase help existing grid-based methods
 - **PEPS break this behavior**



Learning capability study by changing

- Dashed: grid resolution (all scales)
- Line: feature dimension (**only PECS scales**)



Implicit Image Representation

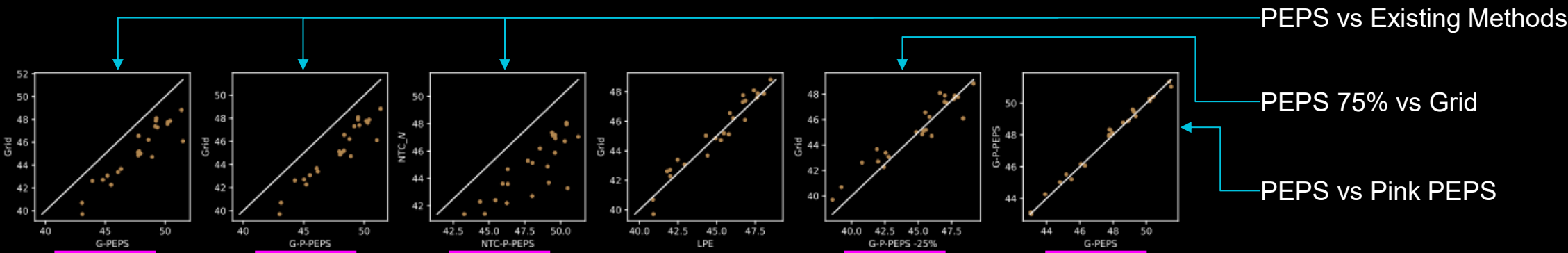
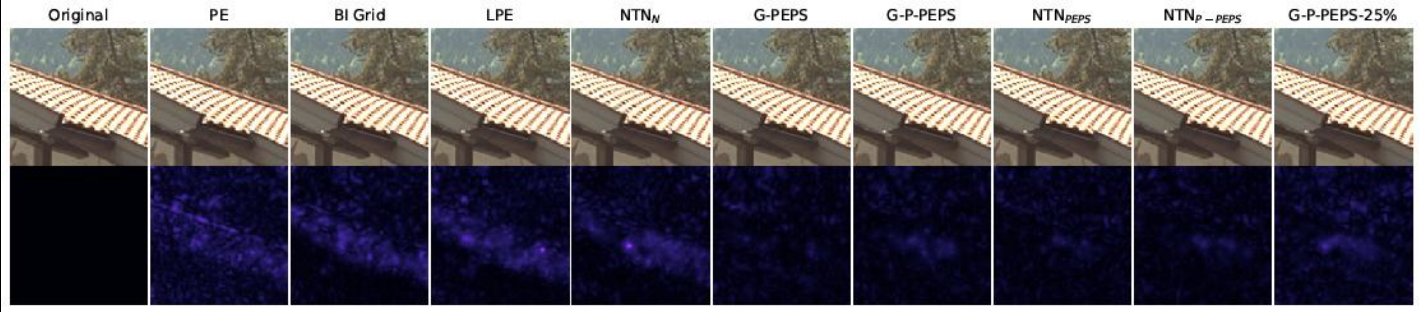
- Problem:
 - Given a coordinate $x \in [0, 1]^2$, the function $P(x)$ returns the RGB values $(r, g, b) \in \mathbb{R}^3$
- Settings:
 - Learning the Kodak dataset (24 images, 768*512)
 - Using 2.7 times less parameters (small compression)

Existing Methods

PEPS

PEPS – 25%

Method	PSNR ($\uparrow$)	FLIP ($\downarrow$)	LPIPS ($\downarrow$)	LSD ($\downarrow$)	SSIM ($\uparrow$)
PE	39.91	3.71e-02	1.47e-02	4.46e-02	0.960
LPE -6%	45.06	1.84e-02	1.62e-03	7.49e-03	0.992
NTC _N	44.87	1.96e-02	2.23e-03	9.74e-03	0.992
Grid	45.30	1.83e-02	1.37e-03	1.24e-02	0.992
G-PEPS	47.72	2.08e-02	1.30e-03	4.19e-03	0.993
G-P-PEPS	47.83	2.24e-02	1.05e-03	4.41e-03	0.993
NTC _{PEPS}	48.02	2.07e-02	1.43e-03	4.07e-03	0.994
NTC _{PinkPEPS}	48.07	2.14e-02	1.20e-03	4.22e-03	0.994
G-P-PEPS -25%	44.89	2.81e-02	3.12e-03	8.45e-03	0.987





Neural Texture Compression

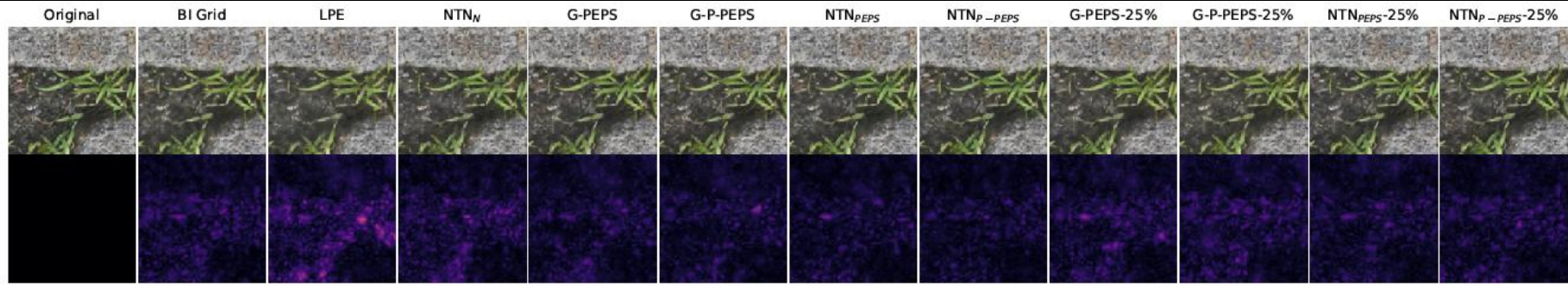
- Problem:
 - Given a coordinate $x \in [0, 1]^2$, the function $T(x)$ returns the texture $t \in \mathbb{R}^{3k}$
- Settings:
 - Grid methods: resolution 1024, features 17
 - NTC methods: 2 grids
 - resolution 1024, features 12
 - resolution 512, features 20
 - Tiled positional encoding
- Results:
 - PEPS usually yields better quality results
 - PEPS Models using $\frac{1}{4}$ less parameters are on par with State of the Art

Existing Methods

PEPS

PEPS – 25%

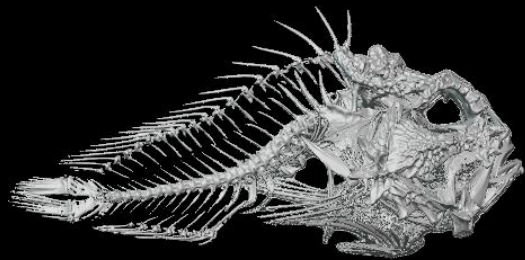
Method	PSNR	SSIM	AO	ARM	DIFF	Disp.	metal	normals	rough	specular
LPE	40.21	0.95	42.16	41.18	36.38	50.34	46.83	34.99	39.05	44.76
NTC _N	40.20	0.95	42.59	41.29	36.78	48.99	46.07	34.99	39.18	46.13
BI grid	41.25	0.95	43.59	42.24	37.42	51.18	47.35	36.03	39.98	46.07
Grid-PEPS4F	41.23	0.95	43.42	42.11	37.09	50.81	49.90	35.90	39.67	47.74
Grid-PinkPEPS4F	41.44	0.95	43.55	42.48	37.48	50.64	49.43	36.24	40.10	47.73
NTC _{PEPS}	41.79	0.95	44.17	42.73	37.91	50.28	50.47	36.64	40.23	48.56
NTC _{PinkPEPS}	41.89	0.95	44.34	42.79	38.09	50.42	50.10	36.71	40.47	48.24
Grid-PEPS4F-25%	39.86	0.93	42.19	40.83	35.86	49.15	48.63	34.58	38.50	45.41
Grid-PinkPEPS4F-25%	40.03	0.94	42.32	41.26	36.20	48.83	48.07	34.81	38.83	45.31
NTC _{PEPS} -25%	40.59	0.94	43.17	41.60	36.70	49.65	49.23	35.27	39.10	46.15
NTC _{PinkPEPS} -25%	40.56	0.94	43.18	41.65	36.74	49.61	47.90	35.30	39.19	46.12



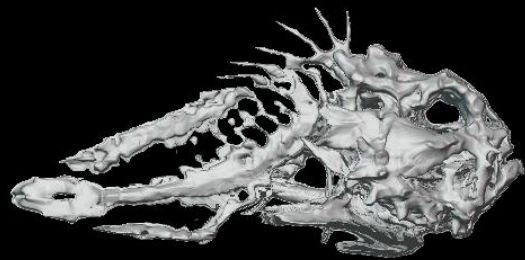


Signed Distance Function

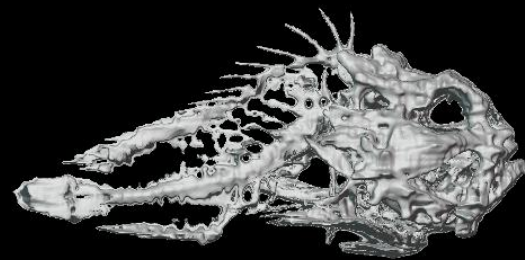
- Problem:
 - Given a coordinate $x \in [0, 1]^3$, the function $S(x)$ returns the distance to surfaces $d \in \mathbb{R}$
- Settings:
 - Four SDF instances are defined using 512^3 values.
 - Grid method: resolution 32, features 18
 - Hash-grid: resolution 64, features 18
 - Multi-grid method: resolutions [16,32,64], features 2
 - Multi-hash-grid method: resolutions [16,32,64,128], features 2
 - MLP is composed of 3 layers of 64 neurons.
 - The compression factor is 227 (ratio is 99.6%).



GT



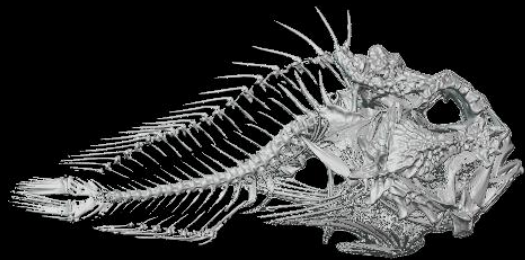
Grid



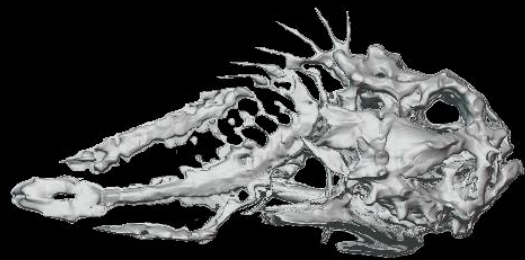
LPE



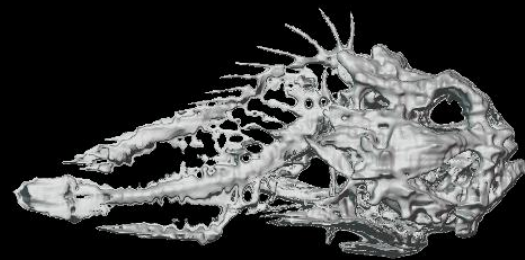
PE



GT



Grid

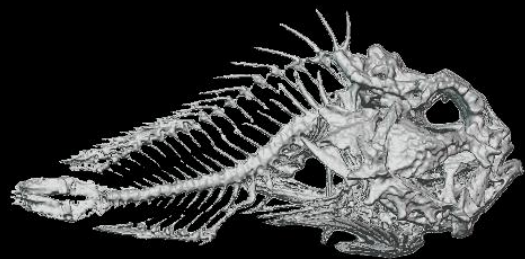


LPE

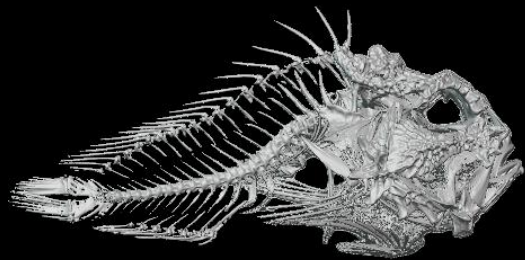


PE

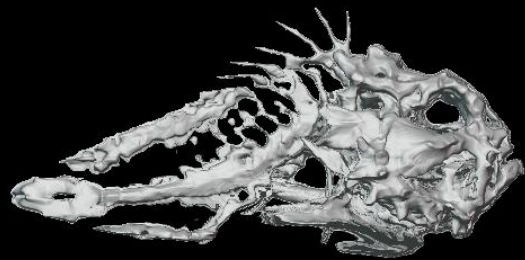
Same Grid



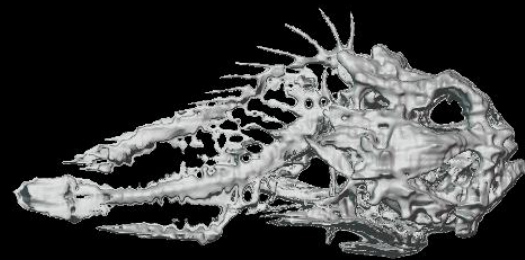
PEPS



GT



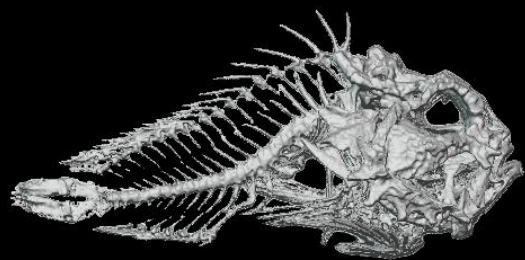
Grid



LPE



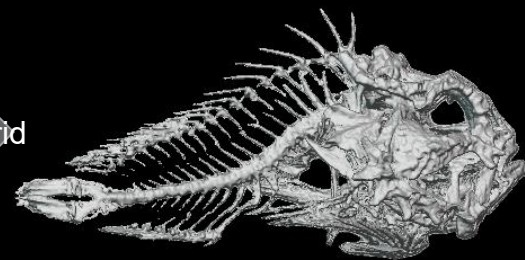
PE



PEPS



MultiGrid

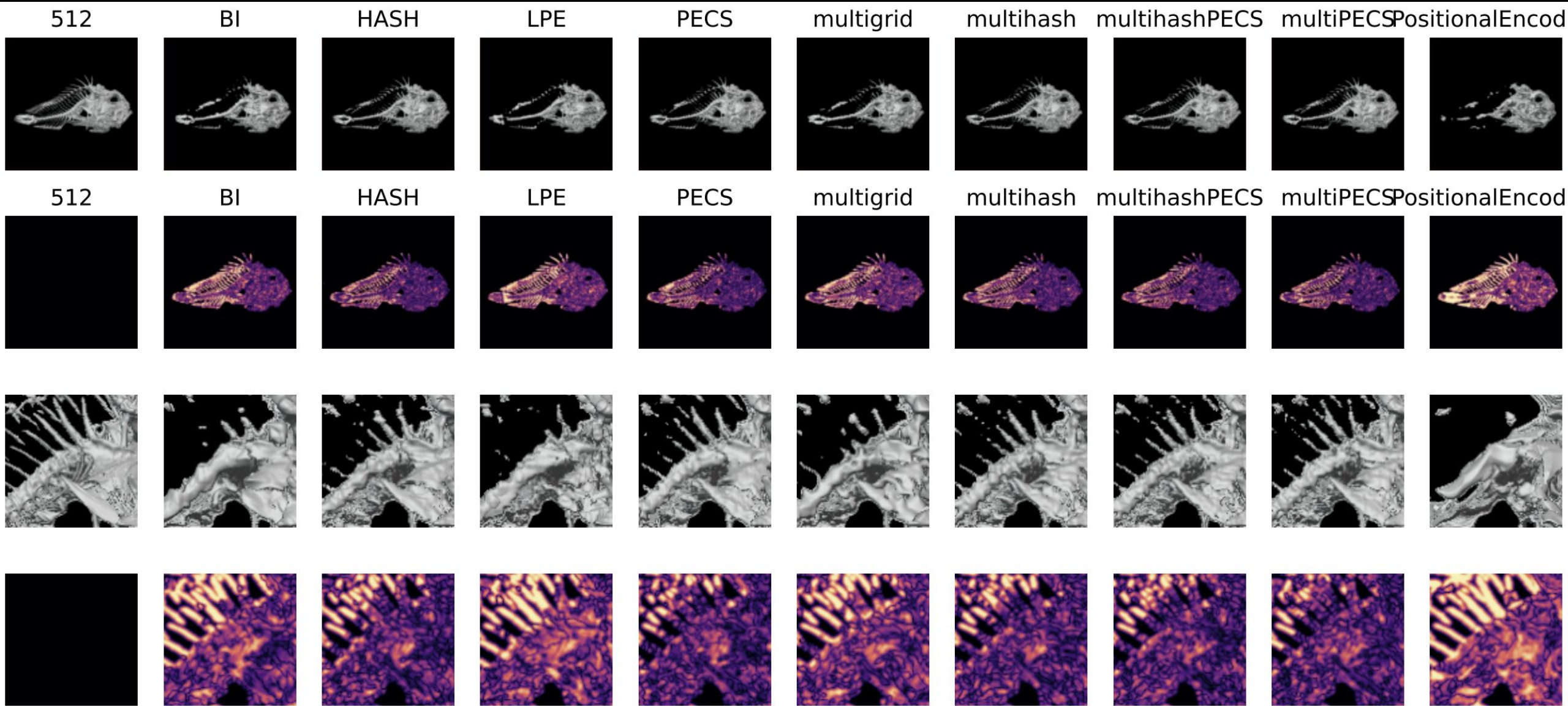


MultiGridPEPS





More Comparison in the Supplemental Material





Signed Distance Function

- Problem:

- Given a coordinate $x \in [0, 1]^3$, the function $S(x)$ returns surfaces $d \in \mathbb{R}$

- Settings:

- Four SDF instances are defined using 512^3 values.
- Grid method: resolution 32, features 18
- Hash-grid: resolution 64, features 18
- Multi-grid method: resolutions [16,32,64], features 2
- Multi-hash-grid method: resolutions [16,32,64,128], features 2
- MLP is composed of 3 layers of 64 neurons.
- The compression factor is 227 (ratio is 99.6%).

- Results:

- The PEPS methods are improving
- Reduce artifact
- Got the quality that other models need **8 times more parameters** to reach

Table 3. (SDF) IoU ($\uparrow$) results for all the methods and our PEPS counterparts. Loss is MAPE.

Model	PE	LPE	TI Grid	Grid-PEPS	Hash	hashPEPS	M-Grid	M-PEPS	M-Hash	M-HashPEPS
Lucy	0.893	0.902	0.904	<i>0.909</i>	0.640	0.876	0.902	0.908	0.910	0.910
Pitted Stonefish	0.316	0.389	0.410	0.470	0.339	0.428	0.413	<i>0.469</i>	0.460	0.470
Thai Statue	0.918	0.925	<i>0.926</i>	0.930	0.574	0.831	0.924	0.930	0.930	0.930
Armadillo	0.952	<i>0.956</i>	0.955	0.957	0.637	0.788	<i>0.956</i>	0.957	0.957	0.957
Global	0.770	0.793	0.799	<i>0.816</i>	0.547	0.731	0.799	<i>0.816</i>	0.814	0.817

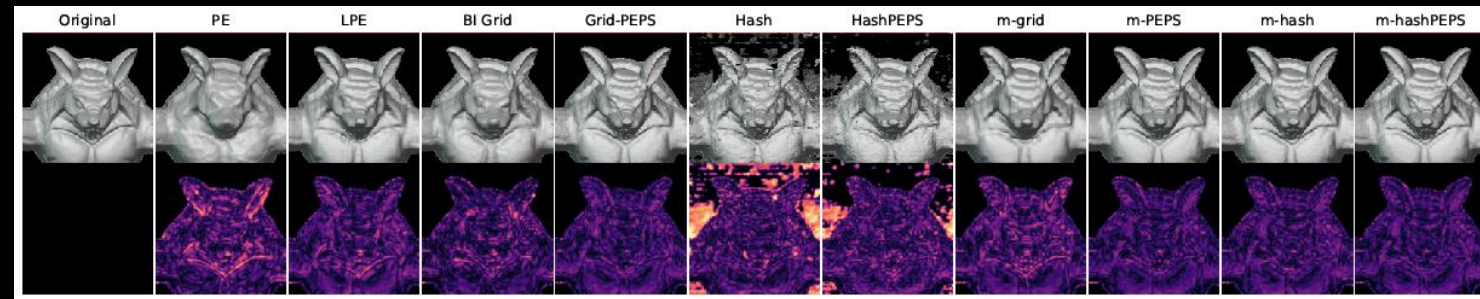


Table 4. (SDF) IoU ($\uparrow$) results for all the methods using L1 loss. The bold line uses 8 times more parameters in the encoders (grid of resolution 64 instead of 32).

Model	Grid	hash	LPE	Grid-PEPS	PE	m-PEPS	m-grid	m-hash	m-hashPEPS
Pitted Stonefish	0.390	0.242	0.363	0.453	0.206	0.446	0.399	0.442	0.444
Pitted Stonefish	0.456	0.350	0.450	0.466	0.206	0.462	0.454	0.466	0.466

More results in the paper



Questions

- Does grid-PEPS break the resolution as core parameter for improvements?
 - YES
- Does PEPS improve reconstruction quality?
 - YES, above the SOA
- Does it reduce model size?
 - 25% reduction
- Does it reduce artifacts and improve details?
 - YES



Limitations, Future Work, Conclusion

- **Conclusion:**

- Introduced PEPS: Positional Encoding Projected Sampling
 - Treats positional encoding as curves, samples along them
- Two implementations:
 - Grid-PEPS: wraps existing grid encoders
 - Grid-Pink-PEPS: spectrally guided, efficient allocation
- Across images, textures, SDFs:
 - Higher quality, fewer artifacts
 - ~25% fewer parameters than strong baselines

- **Limitations:**

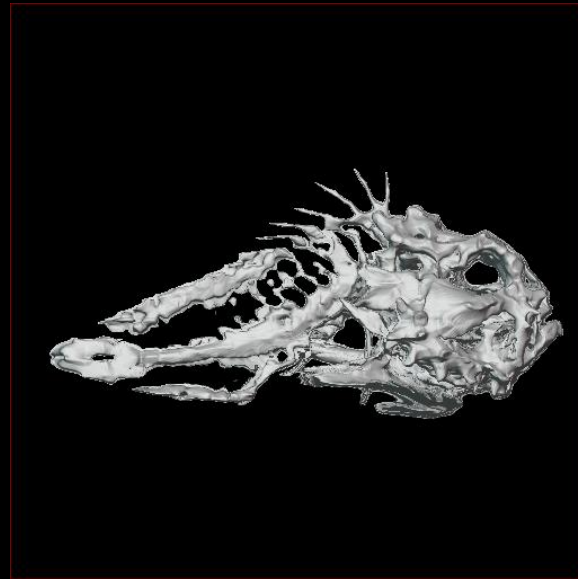
- Overhead vs. simplest grid encoders
 - GridPinkPEPS with 3 frequencies: 12.5 % overhead
 - GridPinkPEPS with 3 frequencies: 15.5 % overhead
- Currently tested on specific tasks and datasets

- **Future directions:**

- Integration into NeRF-like volume rendering
- Extensions to dynamic scenes or temporal signals

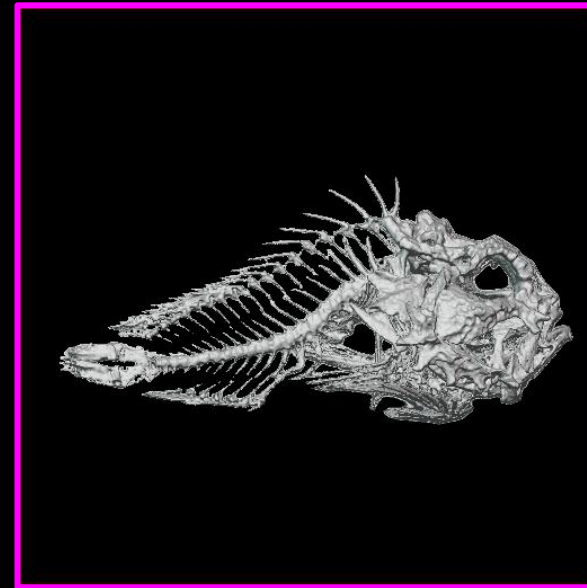


SDF Reconstruction Example



$\text{MLP}(\text{Grid}(X))$

Same Grid
Same # of params



$\text{MLP}(\text{Grid}(\text{PEPS}(X)))$



Pseudo Code

Grid Encoding

```
store( outputBuffer, BilinearInterpolation(u, v) )
```

Grid Encoding + PEPS

```
// 1. Initial Sampling
// Extract base features using the original coordinates (u, v)
store( outputBuffer, BilinearInterpolation(u, v) )
outputBuffer += stride

// 2. Positional Encoding (PEPS) Loop
// Generate features by mapping input coordinates to sine/cosine space
For each frequency f from 0 to numberOfFrequencies:
    theta = PI * (2^f)

    // Transform original coordinates into four new sampling points
    xs = ((sin(src_x * theta) + 1) * 0.5)
    xc = ((cos(src_x * theta) + 1) * 0.5)
    ys = ((sin(src_y * theta) + 1) * 0.5)
    yc = ((cos(src_y * theta) + 1) * 0.5)

    // Sample features at the Sine-based coordinates (xs, ys)
    store( outputBuffer, BilinearInterpolation(xs, ys) )
    outputBuffer += stride

    // Sample features at the Cosine-based coordinates (xc, yc)
    store( outputBuffer, BilinearInterpolation(xc, yc) )
    outputBuffer += stride
```